

temp.py × Terminal

```
1  # WRITE YOUR CODE HERE
2  import openai
3  import os
4
5  openai.api_key='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'
6  message=[{"role": "system", "content": "You are a collaborative programming assistant with expertise in Python. The user
7            {"role": "user", "content": "Alice: I'm having trouble understanding how to use Python decorators, but I
8            {"role": "user", "content": "Bob: I think I understand decorators, but I'm not sure how to use them."}
9            ]
10
11  response=openai.ChatCompletion.create(
12      model="gpt-3.5-turbo",
13      messages= message
14  )
15  print(response['choices'][0]['message']['content'].strip())
```

Guide ×

CollapseIntro to GPT-3

<>⚙️☰

Choosing System

Lets try creating a system message that sets the context of the AI as a collaborative programming assistant with expertise in Python. The user messages represent two team members, Alice and Bob, with distinct questions related to Python decorators.

First step in our code we are going to get our libraries and API keys.

```
import os
import openai
import secret
openai.api_key=secret.api_key
```

TRY IT!

Command was successfully executed.

To make visualizing easier, we are going to create a variable named `message` that will store our prompt.

```
message=[{"role": "system", "content": "You are a collaborative programming assistant with expertise in Python. The user
messages represent two team members, Alice and Bob, with distinct questions related to Python decorators."}
{"role": "user", "content": "Alice: I'm having trouble understanding how to use Python decorators, but I
{"role": "user", "content": "Bob: I think I understand decorators, but I'm not sure how to use them."}
]
```

TRY IT!

After we are going to make our API call and set it equal to `response`. We will use slicing with our print statement in order to better see the information that the response generates.

```
response=openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages= message
)
print(response['choices'][0]['message']['content'].strip())
```

TRY IT!

Terminated by timeout

Why does the 'role' attribute matter when using the ChatGPT API?

☒ It differentiates between user and system messages.

☐ It changes the language of the response.

☐ It modifies the speed of the response.

☐ It affects the sentiment of the response.

The 'role' attribute is used to differentiate between user and system messages. User messages instruct the model on what task to perform, and system messages provide important contextual information about the task. This distinction is crucial for the API to generate the correct and contextually appropriate responses.

Check It!

Next

100% (15:60)

https://codio.com/sandipan-dey/collaborative-programming-with-chatgpt-api/tree/temp.py

1/1